C语言程序设计期中复习：第1—4章

使用说明

依据《程序设计》Chapter1、Chapter2、Chapter3、Chapter4课件整理，Chapter0作为少量先修补充。重点标记依据课件例题、练习和讲解篇幅判断，并非对实际考试的保证。本资料使用标准C写法；考试时仍应留意老师对编译环境和代码风格的要求。

- 重点：要求能独立写出、追踪并解释，优先用于程序阅读、改错、填空和编程题。

- 理解：要求能辨析概念和典型错误，重点用于选择题和阅读题。

- 阅读代码时，先看类型与初值，再按执行顺序跟踪变量；数组和循环特别注意边界。

一、按题型复习

题型 主要能力 复习动作

单选 语法、类型、表达式值、错误辨析 手算表达式，检查类型、优先级与边界

程序阅读 输入输出和执行过程 列变量跟踪表，标出每次循环的条件、更新和输出

程序改错 发现语法、逻辑和边界错误 检查编译问题、初始化、条件、更新、输入输出、越界

程序填空 理解已给算法的状态和不变式 先确定变量含义，再确定循环区间和退出条件

编程 独立设计算法并实现 明确输入输出、数据范围、初始状态、步骤和边界样例

二、程序与C语言基础

2.1 基本概念

- 理解：算法是解决问题的有限、明确的步骤；数据结构描述数据及其关系。算法要正确、清晰，并考虑效率。

- 程序由顺序、选择、循环三种基本控制结构及其嵌套构成。

- 开发过程：分析问题、设计算法、编写代码、编译链接、运行测试、调试。编译成功不等于结果正确。

- 源文件先经预处理和编译，再链接所需库，生成可执行程序。

2.2 基本程序骨架

```
#include <stdio.h>

int main(void) {
    int x, y;
    if (scanf("%d%d", &x, &y) != 2) return 1;
    printf("%d\n", x + y);
    return 0;
}
```

- #include 是预处理指令，通常独占一行且不加分号；stdio.h 提供标准输入输出函数的声明。

- main 是程序入口，return 0 表示正常结束；语句一般以分号结束，复合语句用 {} 包围。

- 标识符通常由字母或下划线开头，后接字母、数字、下划线；区分大小写，不能使用关键字。避免自己定义以下划线开头的名称。

- 常量的值不变，变量先声明再使用；局部变量未初始化时不能假设其值为0。

- #define N 100 定义符号常量，不写结尾分号；注意宏只是预处理替换。

三、基本数据与表达式

3.1 整数、浮点与字符

- 课件补充：进制与存储：8 bit = 1 byte；按二进制容量计算，1 KiB = 1024 byte。二进制每3位可对应1位八进制、每4位对应1位十六进制；例如二进制 1100100 等于十进制100、八进制144、十六进制64。

- 固定位数的补码：正数按二进制表示；负数可将其正数二进制逐位取反后加1。例如在8位补码中，11为 00001011，-11为 11110101。做题必须先明确位宽。

- 浮点常量可以采用科学计数法，例如 2E-5；指数部分必须是整数且不能省略数字。浮点输出 %f 默认保留6位小数；%+d 为正数显示正号，字符串输出 %.3s 最多输出3个字符。

- 重点：int、short、long、unsigned 的范围依实现而定；C保证 short 至少16位、long 至少32位，且 short 不宽于 int、int 不宽于 long。不要默认所有机器上的 int 都是4字节。

- 二进制、八进制和十六进制：012 是八进制10，0x12 是十六进制18。整数后缀如 U、L；整数常量的实际类型还取决于值和书写形式。

- 有符号整数常见地使用补码表示，但计算时必须防止溢出；有符号整数溢出在C中是未定义行为，无符号运算按其范围取模。

- float、double、long double 表示有限精度的实数。3 是整数常量，3.0 默认是 double，3.0f 是 float。

- 十进制小数不一定能精确表示为二进制浮点数。比较近似结果时可用 fabs(a - b) < eps；误差阈值要依据数值规模及题目要求选择，必要时同时考虑绝对和相对误差。fabs 声明于 <math.h>。

- 'A' 是字符常量，"A" 是字符串；'1' 是数字字符，1 是整数。'7' - '0' 得整数7；将0至9的整数变回数字字符可加 '0'。

- 常见转义字符：'\n' 换行、'\t' 制表、'\\' 反斜杠、'\'' 单引号、'\0' 字符串结束符。ASCII中英文字母及数字字符常用于课件中的字符处理。

3.2 运算与转换：阅读题重点

运算 需要掌握

+ - * / % 整数除法向零截断：5/2 为2，-5/2 为-2；% 只用于整数，-5%3 为-2；除数不可为0

++x、x++ 都使x加1；前者表达式值是增加后的值，后者是增加前的值

=、+= 等 赋值有表达式值；x *= a + b 相当于 x = x * (a + b) 的效果

关系与逻辑 < <= > >= == != 的结果为0或1；!、&&、 使用真值，后两者短路

?: 条件 ? 表达式1 : 表达式2，只计算所选分支

逗号运算符 从左向右求值，结果为最后一个表达式的值；注意函数参数之间的逗号不是逗号运算符

sizeof 得到对象或类型的字节大小；例如 sizeof(int)，具体结果依平台而定

- 常见优先顺序：括号、单目、乘除余、加减、关系、相等、&&、 、条件、赋值、逗号。复杂表达式用括号清楚说明意图。

- = 是赋值，== 是比较；1 < x && x < 5 才表示区间，1 < x < 5 会先计算 1 < x，再将0或1与5比较。

- char、short 通常先进行整数提升；混合类型运算按C的转换规则处理，赋值也可能发生转换或精度损失。(double)a / b 能在除法前转换；(double)(a / b) 已经先做整数除法。

- 不要编写或试图预测 i++ + i++ 这类对同一对象无序修改的表达式；它是未定义行为，不存在标准规定的唯一结果。

例： int i = 4; int a = i++; int b = ++i; 执行后 a=4, b=6, i=6。

四、输入输出

4.1 格式输出

- 重点：printf 格式说明与参数数量、类型匹配。%d 输出 int，%c 输出字符，%f 输出浮点数，%s 输出字符串；%e、%g、%o、%x 分别用于科学计数法、紧凑浮点、八进制、十六进制。无符号转换应传相应的无符号类型。

- printf("%6.2f", x)：至少6列，保留2位小数；%-6d 左对齐，%04d 可用零填充。宽度是最小宽度，不会截断过长的数字。

- printf 不自动换行；字符串中的普通字符原样输出，\n 才换行。

4.2 格式输入及字符输入

对象 常用输入 注意

int x scanf("%d", &x) 参数为地址

long x scanf("%ld", &x) 类型必须匹配

float x scanf("%f", &x) 与 double 区别

double x scanf("%lf", &x) long double 用 %Lf

char c scanf("%c", &c) 原样读一个字符，包括空白

char s[20] scanf("%19s", s) 跳过前导空白，读到空白为止，预留 \0

- 重点：scanf(" %c", &c) 中的空格使输入先跳过空白；没有该空格时，前一次输入留下的换行可能被 %c 读入。

- 格式串中的普通字符须与输入匹配：scanf("%d,%d", &a, &b) 要求两个数字间有逗号。%3d 限制最多读取3个字符，%*d 读入但不赋值。

- 检查返回值：scanf("%d", &x) == 1 才表示成功赋值；多个输入检查实际成功项数。输入结束或失败时不能继续使用未赋值的数据。

- getchar() 逐字符读取，空格、换行也算字符；用 int c 保存返回值，才能可靠地区分字符和 EOF。putchar(c) 输出一个字符。

- 阅读输入题时，写出尚未读取的字符序列，逐个模拟格式项；例如输入 12 A，%d%c 的字符得到空格，而 %d %c 得到 A。

五、选择与循环

5.1 选择结构

- 重点：if (条件) 语句; else 语句; 中非零为真；多个语句要用 {} 包围。else 匹配最近的尚未匹配的 if。

- 互斥多分支使用 if ... else if ... else。后续分支可利用前面条件已为假的事实，避免重复判断。

- switch 的控制表达式须为整数类型，case 后为互不重复的整数常量表达式；break 结束当前 switch，省略时会继续执行后续分支。default 处理未匹配情况。

- 易错：if (x = 3) 是赋值后判断，不是相等比较；分号误写在 if (...) 后会形成空语句。

5.2 循环结构及跟踪

```
int sum = 0;
for (int i = 1; i <= n; ++i) {
    sum += i;
}
```

- 重点：while 先判断后执行，可能一次也不执行；do ... while 先执行后判断，至少执行一次，结尾须有分号。

- for (初始化; 条件; 更新) 的执行次序：初始化一次 → 判断条件 → 循环体 → 更新 → 再判断。省略条件表示条件始终为真。

- break 跳出其所在的最近一层循环或 switch；continue 跳过本轮剩余语句。for 中继续执行更新表达式；while 中直接重新判断条件，可能因更新被跳过造成死循环。

- 阅读题画表：列出循环变量、累计量、条件真假和每次输出；注意先输出后更新与先更新后输出不同。

- 设计循环先明确：初始状态、每轮处理对象、更新方式、何时停止、结束后结果含义。检查0次、1次、最后一次循环。

- 嵌套循环中每一轮外层通常都重新执行完整内层；内层 break 只退出内层。

5.3 常见算法模型

累加和计数：和从0开始，乘积通常从1开始；平均值计算前检查计数不为0，必要时用浮点除法。

逐位处理非负整数：digit = n % 10; n /= 10;。数字0是否需要被处理一次由题意决定；负数要先明确范围和处理规则，避免直接对最小整数取负导致溢出。

素数判断：小于2不是素数；从2开始试除，检查到 i <= n / i 可避免 i * i 溢出；找到因子即不是素数。

```
int prime = n >= 2;
for (int i = 2; prime && i <= n / i; ++i) {
    if (n % i == 0) prime = 0;
}
```

最大值与位置：用第一个有效输入初始化，而不是假设最大值为0；用 > 保留第一次出现的位置，用 >= 保留最后一次。次大值须先明确是否允许与最大值相同。

级数与递推：维护当前项、累计和、序号及符号；由上一项推出下一项可减少重复计算和中间溢出。先明确“误差项是否计入”的停止规则。

枚举与图案：明确每层循环代表什么、各范围是否包括端点；利用约束减少无效枚举。图案按行分析空格数和符号数。

课件典型题：菱形逐行输出时，上半部分第 i 行有 n-i 个空格、2*i+1 个星号；下半部分可让 i 从 n-1 递减至0。递推累加 1+(1+2)+...+(1+...+n) 时，先 p += i 再 s += p。枚举鸡兔数量时可利用总数计算另一数量，减少一层循环。

六、数组与常用算法

6.1 一维数组

- 重点：int a[100]; 下标合法范围为 0 到 99；数组容量100不代表当前已有100个有效元素。访问越界是未定义行为。

- int a[5] = {1, 2}; 后面元素补零；int a[5] = {0}; 全部为零。未初始化的局部数组不能直接读取。int a[] = {1, 2, 3}; 根据初值确定长度。

- 通常遍历有效元素：for (i = 0; i < n; ++i)。普通数组不能用 a = b 整体赋值或用 == 比较内容。

- 课件建议使用固定容量，不依赖变长数组；输入 n 时检查 0 <= n && n <= 容量。

插入与删除：假设容量为 CAP，有效长度为 n。

```
/* 在 k 插入 x，要求 0 <= k <= n 且 n < CAP */
for (int i = n; i > k; --i) a[i] = a[i - 1];
a[k] = x;
++n;

/* 删除下标 k，要求 0 <= k < n */
for (int i = k; i < n - 1; ++i) a[i] = a[i + 1];
--n;
```

插入要从后向前移动，避免覆盖尚未移动的数据；删除后最后一个旧值虽然仍在内存，但不再属于有效数据。

6.2 查找

- 顺序查找：从头逐个比较，找到则返回下标，否则检查到 i == n；最坏需检查n个元素。哨兵法需事先留出额外位置 a[n]，且最后仍须区分真正找到与碰到哨兵。

- 成功查找且每个元素被查概率相等时，顺序查找平均比较 (n+1)/2 次；未找到时需比较 n 次。这一平均值依赖上述概率假设。

- 重点：二分查找仅适用于有序数组。下面采用闭区间 [left, right]：

```
int left = 0, right = n - 1, pos = -1;
while (left <= right) {
    int mid = left + (right - left) / 2;
    if (a[mid] == key) { pos = mid; break; }
    if (a[mid] < key) left = mid + 1;
    else right = mid - 1;
}
```

检查空数组、仅一个元素、目标在两端或不存在。半开区间 [left, right) 也可行，但其初始右界、判断和更新规则必须一致。重复值时，上述算法不保证找到第一个或最后一个。

半开区间写法：初始 left=0, right=n，条件 left < right；当 a[mid] < key 时设 left=mid+1，当 a[mid] > key 时设 right=mid。不能混用闭区间的右边界更新。

6.3 冒泡排序与分区

```
for (int i = 1; i < n; ++i) {
    int swapped = 0;
    for (int j = 0; j < n - i; ++j) {
        if (a[j] > a[j + 1]) {
            int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
            swapped = 1;
        }
    }
    if (!swapped) break;
}
```

- 重点：每轮相邻比较后，未排序部分的最大值移到末尾；共最多 n-1 轮。某轮无交换则可提前结束。另一种优化是记录最后交换的位置，缩小下一轮范围。

- 将负数、零、正数分三段：维护 [0, low) 为负、[low, i) 为零、[i, high) 未处理、[high, n) 为正。遇正数与 high-1 交换后，i 不增加，因为新换来的值还未检查。

- 出现次数统计可用计数数组：先初始化为零，再将数据映射到合法下标；仅当数据范围适合时使用。报数出列可用状态数组记录是否已出列，(i + 1) % n 实现循环下标。

6.4 二维数组

- int a[3][4] 有3行4列，访问 a[i][j]；按行连续存放，列下标变化较快。

- int a[2][3] = {{1, 2}, {3}}; 未列出的元素补零。初始化时可省略第一维，但后续维度要明确，例如 int a[][3] = {{1,2,3},{4,5,6}};。

- 使用两层循环按行读写；行列范围独立检查。矩阵题关注每个位置与行号、列号的关系及取模下标。

七、字符数组与字符串

7.1 核心区别

- 重点：C字符串是以 \0 结尾的字符序列；char s[5] = "abc"; 存有 a b c \0，容量5，有效长度3。

- char a[] = {'a', 'b'}; 不是字符串，不能用 %s、strlen 等以字符串方式处理；char b[] = "ab"; 含结束符。

- strlen(s) 统计第一个 \0 之前的字符数；sizeof(s) 在数组定义所在作用域得到整个数组的字节数。strlen 不包含结束符，sizeof 包含数组全部容量。

- 字符数组之间不能直接整体赋值；比较字符串内容用 strcmp，不能用 == 判断内容相等。

- 遍历字符串：for (int i = 0; s[i] != '\0'; ++i)。逐字符构造完字符串后必须写入结束符，并留够空间。

7.2 字符串输入输出和函数

- scanf("%19s", s) 读入一个不含空白的单词；fgets(s, sizeof s, stdin) 可读入空格，最多读取容量减1个字符并补结束符，成功时可能保留换行。输入失败时要检查返回值。

- printf("%s", s) 输出字符串，puts(s) 输出字符串后另加换行。绝不可对没有结束符的字符数组使用 %s。

- <string.h>：strcpy(dst, src) 复制含结束符的字符串；strcat(dst, src) 连接；两者都要求目标空间足够。strcmp(a,b) 返回负、零、正，分别表示前者小于、等于、大于后者。

- strncpy(dst, src, n) 在源长度达到或超过n时不会自动添加结束符；必须确保目标空间和结束符。不能把它简单当作永远安全的复制函数。

- 课件提到的 gets 已从C11移除且不安全；strlwr、strupr 非标准C。使用 fgets；需要大小写转换时可用 <ctype.h> 中的 tolower、toupper，传入 EOF 或可表示为 unsigned char 的值。

7.3 高频字符串题

- 字母计数：先判断字符在对应字母范围，再以 ch - 'A' 或 ch - 'a' 转换为下标，计数数组初始化为0。大小写是否合并由题意决定。

- 单词计数：按题目规定判断单词字符，维护 inword。从非单词字符进入单词字符时计数加1；离开单词时重置状态。测试空串、多个分隔符、末尾无分隔符。

- 最长字符串：逐行读入，检查结束条件，比较长度，必要时复制当前字符串；考虑首行为空、相同长度如何处理、fgets 保留换行。

八、改错与填空检查表

1. 声明和格式：变量是否定义并初始化？scanf 是否给出正确地址和类型？格式符与参数是否匹配？

2. 条件：= 与 == 是否混淆？区间端点是否正确？&& 和 是否按题意组合？

3. 循环：循环变量是否更新？continue 是否跳过更新？首尾是否漏掉或多算一次？是否有死循环？

4. 数组：是否从0开始？最后一次访问是否越界？数组容量和实际长度是否区分？插入移动方向是否正确？

5. 字符串：是否有 \0？容量是否足够？是否错误地用 == 或 = 比较、复制整个字符串？

6. 数值：是否发生整数除法、除零、整数溢出或不适当的浮点比较？是否将字符编码误当数字？

7. 输入与边界：是否检查输入成功？空数据、单元素、重复值、全负数、0和最大范围是否成立？

九、自测清单

不看答案时，应能完成以下任务，并用边界数据验证：

1. 手算带有类型转换、前后自增和短路逻辑的表达式；解释每个变量的最终值。

2. 给定 scanf 格式和一行输入，说明读入的值与剩余字符。

3. 追踪包含 if、switch、嵌套循环、break、continue 的程序输出。

4. 独立编写整数各位之和、素数判断、累加平均值、最大值及其位置；说明特殊输入如何处理。

5. 独立实现数组插入删除、顺序查找、闭区间二分查找和冒泡排序，并解释边界。

6. 独立完成二维数组遍历、字母频率统计和基于状态变量的单词计数。

7. 对每个程序至少测试：普通情况、最小规模、边界值、重复值，以及题目允许的无效或空输入。

十、课件中需要留意的细节

- Chapter0关于UTF-8的描述不准确：现代UTF-8编码使用1至4字节，常见汉字通常为3字节。

- Chapter2中 scanf 的 long double 格式应为 %Lf，不是 %lf。

- Chapter4提到的 gets、strlwr、strupr 不适合作为可移植的标准C方案，参见第七节。

- 二分查找最坏时间复杂度为 O(log n)；具体比较次数和平均次数取决于数组长度、实现与输入分布，课件中的简化公式不应视为普遍精确公式。

来源

- /Users/paperlover/Downloads/程序设计/Chapter0.pdf（先修基础）

- /Users/paperlover/Downloads/程序设计/Chapter1.pdf

- /Users/paperlover/Downloads/程序设计/Chapter2_20260919.pdf

- /Users/paperlover/Downloads/程序设计/Chapter3 2.pdf

- /Users/paperlover/Downloads/程序设计/Chapter4.pdf